

```

package Tienda;

import java.util.Arrays;

public class productcatalog {
    private int[] id;
    private String[] name;
    private double[] price;
    private int[] stock;

    public productcatalog(int[] id, String[] name, double[] price,
int[] stock) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.stock = stock;
    }

    public void resumen() {
        System.out.println("productcatalog{" +
            "id=" + Arrays.toString(id) +
            ", name=" + Arrays.toString(name) +
            ", price=" + Arrays.toString(price) +
            ", stock=" + Arrays.toString(stock) +
            '}');
    }

    public int indexOf(int buscado) {
        for (int i = 0; i < id.length; i++) {
            if (id[i] == buscado) {
                return i;
            }
        }
        return -1;
    }

    public int indexOfId(int buscado) {
        return indexOf(buscado);
    }

    public int indexOdNameContains(String text) {
        for (int i = 0; i < name.length; i++) {
            text = text.toLowerCase();
            if (name[i].toLowerCase().contains(text)) {
                return i;
            }
        }
    }
}

```

```

        return -1;
    }

    public void bubbleSort() {
        boolean ordenado = false;

        while (!ordenado) {
            ordenado = true;
            for (int i = 0; i < id.length - 1; i++) {
                if (id[i] > id[i + 1]) {
                    int temp = id[i];
                    id[i] = id[i + 1];
                    id[i + 1] = temp;

                    String temporal = name[i];
                    name[i] = name[i + 1];
                    name[i + 1] = temporal;

                    double temporal2 = price[i];
                    price[i] = price[i + 1];
                    price[i + 1] = temporal2;

                    temp = stock[i];
                    stock[i] = stock[i + 1];
                    stock[i + 1] = temp;

                    ordenado = false;
                }
            }
        }
    }

    public int bynarySearch(int idBuscado, boolean isOrdenado) {
        if (isOrdenado) {
            int low = 0;
            int high = id.length - 1;
            while (low <= high) {
                int mid = low + (high - low) / 2;
                if (id[mid] == idBuscado) {
                    return mid;
                }
                if (id[mid] > idBuscado) {
                    high = mid - 1;
                }
                else if (id[mid] < idBuscado) {

```

```

        low = mid + 1;
    }
}
return -1;
}

public int binarySearchId(int idBuscado, boolean idsOrdenados)
{
    return bynarySearch(idBuscado, idsOrdenados);
}

public int getStock(int idx) {
    if (idx < 0 || idx >= stock.length) {
        throw new IllegalArgumentException("Index negativo");
    }
    return stock[idx];
}

public int getStockByIndex(int idx) {
    return getStock(idx);
}

public double getPriceByIndex(int idx) {
    if (idx < 0 || idx >= price.length) {
        throw new IllegalArgumentException("Index negativo");
    }
    return price[idx];
}

public boolean decreaseStock(int idx, int qty) {
    int stockAtual = getStock(idx);
    if (stockAtual >= qty) {
        stock[idx] -= qty;
        return true;
    }
    return false;
}

public void printProductDetails(int idx) {
    System.out.println("Producto: " +
        id[idx] + " . " + name[idx]
    );
}

public void printProductDetail(int idx) {
    printProductDetails(idx);
}

```

```

    public int getSize() {
        return id.length;
    }

    public String getProductNameByIndex(int idx) {
        if (idx < 0 || idx >= name.length) {
            throw new IllegalArgumentException("Índice inválido");
        }
        return name[idx];
    }

    public int getProductIdByIndex(int idx) {
        if (idx < 0 || idx >= id.length) {
            throw new IllegalArgumentException("Índice inválido");
        }
        return id[idx];
    }

    public void printCatalog() {
        System.out.println("ID\tNOMBRE\tPRECIO\tSTOCK");
        for (int i = 0; i < id.length; i++) {
            System.out.println(id[i] + "\t" + name[i] + "\t" +
price[i] + "\t" + stock[i]);
        }
    }
}

```

```
package Tienda;
```

```
/*
```

```
cartId[]
```

```
cartQty[]
```

```
cartSize
```

```
Constructor: con capacidad carrito
```

```
*/
```

```

public class Cart {
    private int[] cartId;
    private int[] cartQty;
    private int cartSize;

    public Cart(int capacidadCarrito) {

```

```

        this.cartSize = 0;
        this.cartId = new int[capacidadCarrito];
        this.cartQty = new int[capacidadCarrito];
    }
    public int indexOfCartId(int idbuscado) {
        for (int i = 0; i < cartSize; i++) {
            if (cartId[i] == idbuscado) {
                return i;
            }
        }
        return -1;
    }
    public void add(int productId, int qty, productcatalog catalog
) {
        if (cartSize >= cartId.length) {
            System.out.println("El carrito esta lleno");
            return;
        }
        if (qty < 0) {
            throw new IllegalArgumentException("Cantidad
negativa");
        }
        int posicion = indexOfCartId(productId);
        if (posicion == -1) {
            throw new IllegalArgumentException("No existe el
producto en el catalogo");
        }

        int indiceCarrito = indexOfCartId(productId);
        int stockActual = catalog.getStock(productId);
        if (indiceCarrito == -1) {
            if (stockActual < qty) {
                throw new IllegalArgumentException("No hay
stock");
            }
            cartId[cartSize] = productId;
            cartQty[cartSize] = qty;
            cartSize++;
        } else {
            if (stockActual < (cartQty[indiceCarrito] + qty)) {
                throw new IllegalArgumentException("");
            }
            cartQty[indiceCarrito] += qty;
        }
    }

    public void remove(int id) {
        int idx = indexOfCartId(id);

```

```

        if (idx == -1) {
            throw new IllegalArgumentException("No existe el
producto en el carrito");
        }
        for (int i = idx; i < cartSize - 1; i++) {
            cartId[i] = cartId[i + 1];
            cartQty[i] = cartQty[i + 1];
        }
        cartId[cartSize - 1] = 0;
        cartQty[cartSize - 1] = 0;
        cartSize--;
        System.out.println("Producto eliminado del carrito");
    }
    public void clear() {
        for (int i = 0; i < cartSize ; i++) {
            cartId[i] = 0;
            cartQty[i] = 0;
        }
        cartSize = 0;
        System.out.println("Carrito vaciado");
    }
    public boolean isEmpty() {
        return cartSize == 0;
    }
}

public int[] getCartId() {
    return cartId;
}

public int[] getCartQty() {
    return cartQty;
}

public int getCartSize() {
    return cartSize;
}
}

package Tienda;

public class SalesLog {
    private int[][] sales;
    private int salesSize;

    public SalesLog(int capacidad) {
        this.sales = new int[capacidad][2];
        this.salesSize = 0;
    }
}

```

```

public void addSale(int productId, int qty) {
    if (salesSize >= sales.length) {
        throw new RuntimeException("Matriz de ventas llena");
    }

    sales[salesSize][0] = productId;
    sales[salesSize][1] = qty;
    salesSize++;
}

public void printSalesTable(productcatalog catalog) {
    if (salesSize == 0) {
        System.out.println("No hay ventas registradas");
        return;
    }

    System.out.println("VENTAS REGISTRADAS:");
    for (int i = 0; i < salesSize; i++) {
        int idx = catalog.indexOfId(sales[i][0]);
        if (idx != -1) {
            String nombre = catalog.getProductNameByIndex(idx);
            System.out.println((i+1) + " - ID: " + sales[i][0]
+ " - " + nombre +
                " - Cantidad: " + sales[i][1]);
        }
    }
}

public int[] computeUnitsSoldPerProduct(productcatalog catalog)
{
    int[] soldUnits = new int[catalog.getSize()];

    for (int i = 0; i < salesSize; i++) {
        int idx = catalog.indexOfId(sales[i][0]);
        if (idx != -1) {
            soldUnits[idx] += sales[i][1];
        }
    }

    return soldUnits;
}

public void printTopSeller(productcatalog catalog) {
    int[] soldUnits = computeUnitsSoldPerProduct(catalog);

    int maxIdx = 0;
    for (int i = 1; i < soldUnits.length; i++) {

```

```

        if (soldUnits[i] > soldUnits[maxIdx]) {
            maxIdx = i;
        }
    }

    if (soldUnits[maxIdx] == 0) {
        System.out.println("No hay productos vendidos");
        return;
    }

    System.out.println("Producto más vendido:");
    System.out.println("ID: " +
        catalog.getProductByIdByIndex(maxIdx));
    System.out.println("Nombre: " +
        catalog.getProductByNameByIndex(maxIdx));
    System.out.println("Unidades vendidas: " +
        soldUnits[maxIdx]);
    }
}

package Tienda;

public class OrderService {

    public void confirmOrder(Cart cart, productcatalog catalog,
        SalesLog log) {
        if (cart.isEmpty()) {
            System.out.println("Carrito vacío");
            return;
        }

        printTicket(cart, catalog);

        for (int i = 0; i < cart.getCartSize(); i++) {
            int productId = cart.getCartId()[i];
            int qty = cart.getCartQty()[i];
            int idx = catalog.indexOfId(productId);

            catalog.decreaseStock(idx, qty);

            log.addSale(productId, qty);
        }

        cart.clear();
        System.out.println("Pedido confirmado");
    }

    public void printTicket(Cart cart, productcatalog catalog) {

```

```

        if (cart.isEmpty()) {
            System.out.println("Carrito vacío");
            return;
        }

        System.out.println("TICKET:");
        double total = 0;

        for (int i = 0; i < cart.getCartSize(); i++) {
            int productId = cart.getCartId()[i];
            int qty = cart.getCartQty()[i];

            int idx = catalog.indexOfId(productId);
            if (idx != -1) {
                String nombre = catalog.getProductNameByIndex(idx);
                double precio = catalog.getPriceByIndex(idx);
                double subtotal = precio * qty;
                total += subtotal;

                System.out.println(nombre + " x" + qty + " = " +
subtotal + "€");
            }
        }

        System.out.println("TOTAL: " + total + "€");
    }
}

```

```

package Tienda;

```

```

public class OrderService {

    public void confirmOrder(Cart cart, productcatalog catalog,
SalesLog log) {
        if (cart.isEmpty()) {
            System.out.println("Carrito vacío");
            return;
        }

        printTicket(cart, catalog);

        for (int i = 0; i < cart.getCartSize(); i++) {
            int productId = cart.getCartId()[i];
            int qty = cart.getCartQty()[i];
            int idx = catalog.indexOfId(productId);

            catalog.decreaseStock(idx, qty);
        }
    }
}

```

```

        log.addSale(productId, qty);
    }

    cart.clear();
    System.out.println("Pedido confirmado");
}

public void printTicket(Cart cart, ProductCatalog catalog) {
    if (cart.isEmpty()) {
        System.out.println("Carrito vacío");
        return;
    }

    System.out.println("TICKET:");
    double total = 0;

    for (int i = 0; i < cart.getCartSize(); i++) {
        int productId = cart.getCartId()[i];
        int qty = cart.getCartQty()[i];

        int idx = catalog.indexofid(productId);
        if (idx != -1) {
            String nombre = catalog.getProductNameByIndex(idx);
            double precio = catalog.getPriceByIndex(idx);
            double subtotal = precio * qty;
            total += subtotal;

            System.out.println(nombre + " x" + qty + " = " +
                subtotal + "€");
        }
    }

    System.out.println("TOTAL: " + total + "€");
}

package Tienda;

import java.util.Scanner;

public class Main2 {
    public static void main(String[] args) {
        int[] ids = {101, 205, 150, 310, 404, 120, 333};
        String[] names = {"Pan", "Leche", "Huevos", "Pasta",
            "Queso", "Arroz", "Tomate"};
        double[] prices = {1.20, 1.05, 2.90, 1.30, 3.75, 1.10,
            0.95};
        int[] stock = {20, 15, 8, 25, 6, 30, 18};
    }
}

```

```

        productcatalog catalog = new productcatalog(ids, names,
prices, stock);
        Cart cart = new Cart();
        SalesLog salesLog = new SalesLog(20);
        OrderService orderService = new OrderService();

        Scanner sc = new Scanner(System.in);
        int option;

        do {
            printMenu();
            option = readInt(sc, "Elige opción: ");

            switch (option) {
                case 1 -> catalog.printCatalog();
                case 2 -> searchMenu(sc, catalog);
                case 3 -> {
                    int id = readInt(sc, "ID producto: ");
                    int qty = readInt(sc, "Cantidad: ");
                    try {
                        cart.add(id, qty, catalog);
                        System.out.println("Añadido al carrito.");
                    } catch (Exception ex) {
                        System.out.println("Error: " +
ex.getMessage());
                    }
                }
                case 4 -> cart.printCart(catalog);
                case 5 -> {
                    int id = readInt(sc, "ID a eliminar del carrito:
");
                    try {
                        cart.remove(id);
                    } catch (Exception ex) {
                        System.out.println("Error: " +
ex.getMessage());
                    }
                }
                case 6 -> orderService.confirmOrder(cart, catalog,
salesLog);
                case 7 -> {
                    salesLog.printSalesTable(catalog);
                    salesLog.printTopSeller(catalog);
                }
                case 8 -> System.out.println("Saliendo...");
                default -> System.out.println("Opción inválida.");
            }
        }

```

```

    } while (option != 8);

    sc.close();
}

private static void printMenu() {
    System.out.println("\nMENU");
    System.out.println("1) Ver catálogo");
    System.out.println("2) Buscar producto");
    System.out.println("3) Añadir al carrito");
    System.out.println("4) Ver carrito");
    System.out.println("5) Eliminar del carrito");
    System.out.println("6) Confirmar pedido");
    System.out.println("7) Ver ventas");
    System.out.println("8) Salir");
}

private static void searchMenu(Scanner sc, ProductCatalog
catalog) {
    System.out.println("\nBUSQUEDA");
    System.out.println("1) Por ID");
    System.out.println("2) Por nombre");
    System.out.println("3) Por ID (binaria, requiere
ordenado)");
    int op = readInt(sc, "Elige: ");

    switch (op) {
        case 1 -> {
            int id = readInt(sc, "ID a buscar: ");
            int idx = catalog.indexOfId(id);
            if (idx == -1) System.out.println("No
encontrado.");
            else catalog.printProductDetail(idx);
        }
        case 2 -> {
            System.out.print("Texto: ");
            String text = sc.nextLine();
            int idx = catalog.indexOfNameContains(text);
            if (idx == -1) System.out.println("No
encontrado.");
            else catalog.printProductDetail(idx);
        }
        case 3 -> {
            int id = readInt(sc, "ID a buscar: ");
            catalog.bubbleSort();
            int idx = catalog.binarySearchId(id, true);
            if (idx == -1) System.out.println("No
encontrado.");

```

```
        else catalog.printProductDetail(idx);
    }
    default -> System.out.println("Opción inválida.");
}

private static int readInt(Scanner sc, String mensaje) {
    while (true) {
        System.out.print(mensaje);
        try {
            return Integer.parseInt(sc.nextLine().trim());
        } catch (NumberFormatException ex) {
            System.out.println("Introduce un número.");
        }
    }
}
}
```